

Отчёт по лабораторной работе №2

1. Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2. Задание

Задача 3. Для векторов x и y :

- выберите элементы вектора y , значения которых больше 600, и выведите на экран; определите индексы этих элементов;
- определите значения вектора x , соответствующие значениям вектора y , значения которых больше 600 (под соответствием понимается расположение на аналогичных индексных позициях);
- сформируйте вектор $(|x_1 - \bar{x}|^2, |x_2 - \bar{x}|^2, \dots, |x_n - \bar{x}|^2)$, где $\bar{x}$ обозначает среднее значение вектора $x = (x_1, x_2, \dots, x_n)$;
- определите, сколько элементов вектора y отстоят от максимального значения не более, чем на 200;
- определите, сколько чётных и нечётных элементов вектора x ;
- определите, сколько элементов вектора x кратных 7;
- отсортируйте элементы вектора x в порядке возрастания элементов вектора y ;
- выведите элементы вектора x , которые входят в десятку наибольших (top-10);
- сформируйте вектор, содержащий только уникальные (неповторяющиеся) элементы вектора x .

Задача 4. Составьте массив `squares`, в котором будут храниться квадраты всех чисел от 1 до 100.

Задача 5. Подключите пакет `Primes` (утилиты для вычисления простых чисел). Сгенерируйте массив `primes`, в котором будут храниться первые 168 простых чисел. Определите 89-е наименьшее простое число. Получите срез массива с 89-го по 99-й элемент включительно, содержащий наименьшие простые числа.

Задача 6. Вычислите следующие выражения:

- **6.1)** $\sum_{i=10}^{100} (i^3 + 4i^2)$;
- **6.2)** $\sum_{i=1}^M (2i/i + 3i/i^2)$, $M = 25$;
- **6.3)** $1 + 2/3 + (2/3 \cdot 4/5) + (2/3 \cdot 4/5 \cdot 6/7) + \dots + (2/3 \cdot 4/5 \cdot \dots \cdot 38/39)$.

3. Выполнение работы

3.1. Изучение кортежей

```
[3]: t0 = ()
      favouritelang = ("Python", "Julia", "R")
      x1 = (1, 2, 3)
      x2 = (1, 2.0, "tmp")
      x3 = (a=2, b=1+2)

      length(x2)
      x2[1], x2[2], x2[3]
      c = x1[2] + x1[3]
      x3.a, x3.b, x3[2]
      in("tmp", x2), 0 in x2
```

```
[3]: (true, false)
```

созданы пустой, строковый, целочисленный, разнотипный и именованный кортежи. Продемонстрированы `length()`, доступ по индексу, доступ по имени, сложение элементов, проверка вхождения `in()`.

3.2. Изучение словарей

```
[5]: phonebook = Dict{"Иванов И.И." => ("867-5309", "333-5544"), "Бухгалтерия" => "555-1234"}

      keys(phonebook)
      values(phonebook)
      pairs(phonebook)
      haskey(phonebook, "Иванов И.И.")

      phonebook["Сидоров П.С."] = "555-3344"
      pop!(phonebook, "Иванов И.И.")

      a = Dict{"foo" => 0.0, "bar" => 42.0}
      b = Dict{"baz" => 17, "bar" => 13.0}
      merge(a, b)
      merge(b, a)
```

```
[5]: Dict{String, Real} with 3 entries:
      "bar" => 42.0
      "baz" => 17
      "foo" => 0.0
```

создан словарь `phonebook`. Продемонстрированы `keys()`, `values()`, `pairs()`, `haskey()`, добавление и удаление элементов, объединение словарей через `merge()`.

3.3. Изучение множеств

```
[7]: A = Set([1, 3, 4, 5])
      B = Set("abracadabra")

      S1 = Set([1,2]); S2 = Set([3,4])
      issetequal(S1, S2)

      S3 = Set([1,2,2,3,1,2,3,2,1])
      S4 = Set([2,3,1])
      issetequal(S3, S4)

      C = union(S1, S2)
      D = intersect(S1, S3)
      E = setdiff(S3, S1)
      issetequal(S1, S4)

      push!(S4, 99)
      pop!(S4)
```

[7]: 2

созданы множества.

Продемонстрированы issetequal(), union(), intersect(), setdiff(), issubset(), push!(), pop!().

3.4. Изучение массивов

```
[11]: empty_array_1 = []
      empty_array_2 = (Int64)[]
      empty_array_3 = (Float64)[]

      a = [1, 2, 3]
      b = [1 2 3]

      A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
      B = [[1 2 3] [4 5 6] [7 8 9]]

      c = rand(1, 8)
      C = rand(2, 3)
      D = rand(4, 3, 2)

      roots = [sqrt(i) for i in 1:10]
```

```
[11]: 10-element Vector{Float64}:
       1.0
       1.4142135623730951
       1.7320508075688772
       2.0
       2.23606797749979
       2.449489742783178
       2.6457513110645907
       2.8284271247461903
       3.0
       3.1622776601683795
```

созданы пустые массивы, вектор-столбец, вектор-строка, матрицы, случайные массивы (1D, 2D, 3D), массив через comprehension.

3.5. Общие функции для структур данных

```
[13]: isempty(t0)
      length(x1)
      in(2, x1)
      unique([1,2,2,3,1])
      reduce(+, [1,2,3,4])
      maximum([1,5,3])
      minimum([1,5,3])
```

```
[13]: 1
```

п

родемонстрированы общие для всех структур функции `isempty()`, `length()`, `in()`, `unique()`, `reduce()`, `maximum()`, `minimum()`.

3.6. Векторы x и y

```
[15]: x = rand(1:1000, 50)
      y = rand(1:1000, 50)
```



```
[15]: 50-element Vector{Int64}:
      235
      972
      819
      809
      880
      911
      723
      770
      723
      904
      350
      712
      602
      ⋮
      834
      310
      394
      129
      776
      857
      487
      398
      924
      416
      979
      887
```

сгенерированы два случайных вектора длиной 50.

3.7. решение по подпунктам

3.7.1. Элементы вектора y, значения которых больше 600

```
[19]: big_y = y[y .> 600]
```

```
[19]: 27-element Vector{Int64}:  
      972  
      819  
      809  
      880  
      911  
      723  
      770  
      723  
      904  
      712  
      602  
      876  
      624  
       ⋮  
      980  
      967  
      746  
      834  
      631  
      666  
      834  
      776  
      857  
      924  
      979  
      887
```

3.7.2. Индексы этих элементов

```
[21]: idx = findall(>(600), y)
```

```
[21]: 27-element Vector{Int64}:  
      2  
      3  
      4  
      5  
      6  
      7  
      8  
      9  
     10  
     12  
     13  
     14  
     20  
      ⋮  
     31  
     32  
     34  
     35  
     36  
     38  
     39  
     43  
     44  
     47  
     49  
     50
```

3.7.3. Значения вектора x , соответствующие $y > 600$

```
[23]: x_corresponding = x[idx]
```

```
[23]: 27-element Vector{Int64}:  
      396  
        2  
       46  
      212  
      321  
      138  
      524  
      852  
      137  
      792  
      284  
      408  
      456  
       ⋮  
        9  
      736  
      544  
      705  
      679  
      462  
      426  
      257  
      492  
      120  
      263  
      456
```

3.7.4. Среднее значение вектора x

```
[29]: count(abs.(y .- maximum(y)) .<=200)
```

```
[29]: 17
```

3.7.5. Вектор $(|x_i - \bar{x}|^2)$

```
[27]: x_bar = sum(x) / length(x)
      sq_dev = [(xi - x_bar)^2 for xi in x]
```

```
[27]: 50-element Vector{Float64}:
      3011.8143999999993
      3122.5743999999995
      202392.0144
      164738.57439999998
      57542.414399999994
      17129.574399999998
      98520.6544
      5201.2944000000001
      160096.014400000001
      99149.4144
      200596.4944
      115681.6144
      28183.694399999997
      ⋮
      669.7743999999998
      1859.33440000000004
      101047.6944
      76795.4944
      37978.2144
      1609.61440000000004
      29199.9744
      112976.6544
      110144.33439999999
      56701.1344
      35675.6544
      16.974400000000004
```

3.7.6. Сколько элементов у отстоят от максимума не более чем на 200

```
[31]: even_count = count(iseven, x)
      odd_count = count(isodd, x)
```

```
[31]: 22
```

3.7.7. Количество чётных элементов вектора x

```
[31]: even_count = count(iseven, x)
      odd_count = count(isodd, x)
```

```
[31]: 22
```

3.7.9. Количество элементов вектора x, кратных 7

```
[35]: count(x .% 7 .== 0)
```

```
[35]: 4
```

3.7.10. Индексы сортировки x по возрастанию y

```
[37]: p = sortperm(y)
      x_sorted = x[p]
```

```
[37]: 50-element Vector{Int64}:
      729
      398
      874
      397
      269
      493
       7
      860
      495
      562
      599
       4
      865
       ⋮
      492
      408
      212
      456
      137
      987
      321
      120
      736
      396
      263
       9
```

3.7.12. Top-10 наибольших элементов вектора x

```
[39]: top10 = sort(x, rev=true)[1:10]
```



```
[39]: 10-element Vector{Int64}:
      987
      874
      865
      860
      852
      849
      843
      815
      792
      788
```

3.7.13. Уникальные элементы вектора x

```
[41]: unique_x = unique(x)
```

```
[41]: 49-element Vector{Int64}:  
      397  
      396  
        2  
       46  
      212  
      321  
      138  
      524  
      852  
      137  
        4  
      792  
      284  
        ⋮  
      462  
      426  
      495  
      134  
      729  
      257  
      492  
      281  
      788  
      120  
      690  
      263
```

3.8. Массив квадратов чисел от 1 до 100

```
[43]: squares = [i^2 for i in 1:100]  
      length(squares)  
      squares[1]  
      squares[100]
```

```
[43]: 10000
```

сформирован массив squares из квадратов чисел 1–100.

3.9. Пакет Primes

```
[45]: using Pkg
      Pkg.add("Primes")
```

```
Updating registry at `C:\Users\Asus\.julia\registries\General.toml`
Resolving package versions...
Installed Primes v0.5.7
Installed IntegerMathUtils v0.1.4
Updating `C:\Users\Asus\.julia\environments\v1.12\Project.toml`
[27ebfcd6] + Primes v0.5.7
Updating `C:\Users\Asus\.julia\environments\v1.12\Manifest.toml`
[18e54dd8] + IntegerMathUtils v0.1.4
[27ebfcd6] + Primes v0.5.7
Precompiling packages...
 1392.5 ms ✓ IntegerMathUtils
  771.3 ms ✓ Primes
2 dependencies successfully precompiled in 5 seconds. 348 already pre
compiled.
```

```
[47]: using Primes

primes_arr = primes(1, 1000)[1:168]
slice = primes_arr[89:99]
```

```
[47]: 11-element Vector{Int64}:
      461
      463
      467
      479
      487
      491
      499
      503
      509
      521
      523
```

подключён пакет Primes. Сгенерирован массив primes из первых 168 простых чисел, определено 89-е наименьшее простое число, получен срез с 89-го по 99-й элемент включительно.

3.10. Задача 6.1

```
[49]: sum(i^3 + 4i^2 for i in 10:100)
```



```
[49]: 26852735
```

вычислена сумма $\sum$ от $i = 10$ до 100 ($i^3 + 4i^2$).

3.11. Задача 6.2

```
[51]: M = 25
      sum((2i)/i + (3i)/(i^2) for i in 1:M)
```

```
[51]: 61.44787453326052
```

вычислена сумма $\sum$ от $i = 1$ до M ($2i/i + 3i/i^2$), $M = 25$.

3.12. Задача 6.3

```
[53]: term = 1.0
      total = 1.0
      for k in 2:2:38
          term *= k / (k + 1)
          total += term
      end
      total
```

```
[53]: 6.976346137897618
```

вычислена

сумма $1 + 2/3 + (2/3 \cdot 4/5) + \dots + (2/3 \cdot 4/5 \cdot \dots \cdot 38/39)$.

4. Выводы

В ходе лабораторной работы:

1. Изучены структуры данных, реализованные в Julia: кортежи, словари, множества, массивы.
2. Освоены операции над ними: индексация, доступ по ключу, теоретико-множественные операции, работа с многомерными массивами.
3. Изучены общие функции `isempty()`, `length()`, `in()`, `unique()`, `reduce()`, `maximum()`, `minimum()`.
4. Решены задачи 3–6:
 - обработаны векторы x и y (фильтрация, индексация, сортировка, статистика);
 - сформирован массив `squares` из квадратов чисел от 1 до 100;
 - подключён пакет `Primes`, получены первые 168 простых чисел, найдено 89-е наименьшее простое число, получен срез с 89-го по 99-й элемент;
 - вычислены три выражения из задачи 6.
5. Получены навыки работы с пакетами Julia и оформления результатов в Jupyter Lab.